

claude-windows / edc346c2-92fc-4ad1-b556-c7ab955a5613

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\edc346c2-92fc-4ad1-b556-c7ab955a5613\subagents\workflows\wf_47347652-5ae\agent-af84a49f051956b3e.jsonl`  
- SHA-256: `122a9097a508d416a77f0239925020b900436ff994bd6538bb05bb501de9ee71`  
- Source modified: `2026-07-05T06:58:05+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_47347652-5ae`  
- session_id: `edc346c2-92fc-4ad1-b556-c7ab955a5613`
```

Transcript

1. user (2026-07-05T06:53:50.195Z)

SHARED CONTEXT (verified live this session, 2026-07-05):

```
- Repos (local checkouts = current master):  
  * ENGINE: C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer (GitHub  
  Khelsutra/badminton-highlight-indexer)  
  * SPA: C:\Users\avidu\Projects\khelsutra-guru\khelsutra (web/src is the Studio SPA; GitHub  
  avidullu/khelsutra)  
- The LIVE hosted control-plane is an OLDER build than local master: capabilities reports  
  engine_version "0.1.0",  
  and its capabilities JSON has NO max_upload_mb / max_part_mb fields (current master DOES  
  expose them, added around  
  khelsutra #153). So distinguish "current master code" (what you can read) from "deployed old-  
  build behavior" and  
  note where they likely diverge.  
- Deployed control-plane config.json (base64-injected at boot) sets ONLY:  
  {"deployment":{"profile":"cloud-  
serving","compute_target":"cloud_run"},"indexing":{"skip_ai_handoff":true},  
  "storage":{"backend":"gcs","output_root":"gs://khelsutra-rally-  
serving","input_backend":"gcs",  
  "input_root":"gs://khelsutra-rally-serving/videos"},"security":{"..edge auth..}}  
  Note: it does NOT pin any default/segmenter, and does NOT override security.max_upload_mb or  
  max_part_mb.  
- Live capabilities from the deployed CP: default_segmenter="yolo_hybrid",  
  wasb_weights_present=false,  
  gpu.nvidia_smi_present=false, gemini_key_present=false, deployment.cloud_available=true,  
  profile=cloud-serving.  
- Architecture: the CP (Cloud Run service, 4Gi RAM, no GPU, /tmp is RAM-backed) serves the SPA +  
  dispatches  
  cloud jobs to an L4 GPU Cloud Run JOB "rally-worker" which HAS the WASB weights mounted  
  (WASB_WEIGHTS=gcs/models/...).  
- Config defaults known from code: SecurityConfig.max_upload_mb=4096 (MB whole-file),  
  max_part_mb=95 (per part).  
- The e2e/battery that validated the alpha ALWAYS passed segmenter:"wasb_hybrid" EXPLICITLY in  
  /api/process.
```

ISSUE 1 ? the browser upload of a 2.68 GB (2749 MB) .mp4 fails with HTTP 413 on the hosted alpha, even though

2.68 GB is UNDER the 4096 MB whole-file cap.

Trace the exact server-side 413 path in the ENGINE. Read backend/api/uploads.py (the /upload/init and

/upload/{id}/part/{n} handlers), backend/config/models.py (SecurityConfig max_upload_mb, max_part_mb, and any

free-space / disk-guard config), and any free-space check on the upload staging dir (uploads land under

/tmp/apphome/output/uploads; note /tmp is RAM-backed on the 4Gi Cloud Run instance). Determine WHERE a 2.68 GB upload actually 413s: (a) /upload/init whole-file cap, (b) a per-part >max_part_mb rejection, (c) a free-space pre-check failing because the RAM-backed /tmp can't hold 2.68 GB on a 4Gi instance, or (d) a Cloudflare edge 100 MB body limit on a part (app never sees it). Also state whether the deployed OLD build (engine 0.1.0, no max_upload_mb in capabilities) would behave differently. Give concrete fix options (raise cap, stream-to-GCS resumable upload, bigger instance, direct-to-bucket, etc.).

2. assistant (2026-07-05T06:53:53.972Z)

I'll trace the exact server-side 413 path in the ENGINE. Let me start by pulling the repo and reading the relevant files.

3. user (2026-07-05T06:53:57.583Z)

Already up to date.

4. user (2026-07-05T06:53:58.163Z)

```
backend/api/:
__init__.py
__pycache__
auth.py
job_worker.py
static
uploads.py
```

```
backend/config/:
__init__.py
__pycache__
loader.py
models.py
presets.py
startup.py
```

5. assistant (2026-07-05T06:54:05.280Z)

Let me read the key files: uploads.py, config/models.py, and check for free-space checks.

6. user (2026-07-05T06:54:06.351Z)

```
1      """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3      Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4      proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5      under security.upload_dir, then submitted to /api/process by the returned path.
6      Upload state is in-process by design (this is the single-box alpha): a server restart
7      aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8      arrives with PR-3 identity scoping.
9
10     The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11     ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12     reads the live value the server set at startup (and so tests that monkeypatch
13     ``backend.main.global_config`` flow through unchanged).
14     """
15
16     import os
17     import threading
18     import unicodedata
19     import uuid
```

```

20     from typing import Any, Dict, Optional
21
22     from fastapi import APIRouter, HTTPException, Request
23     from pydantic import BaseModel
24
25     from backend.utils.app_home import (
26         resolve_output_dir, # P0a: app-home-aware path resolution
27     )
28     from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
29     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
30     from backend.utils.paths import safe_output_filename
31
32     router = APIRouter()
33
34     _uploads: Dict[str, Dict[str, Any]] = {}
35     _uploads_lock = threading.Lock()
36
37
38     class UploadInitRequest(BaseModel):
39         filename: str
40         total_size_bytes: Optional[int] = None
41
42
43     class UploadCompleteRequest(BaseModel):
44         total_parts: int
45
46
47     def _upload_cfg():
48         # Resolved lazily (import inside the function) to read the LIVE startup config that
49         # main() sets on the module global, and to avoid a circular import at module load
50         # (main.py imports this router; this module must not import main at import time).
51         import backend.main as _main
52
53         sec = getattr(_main.global_config, "security", None)
54         upload_dir = resolve_output_dir(
55             getattr(sec, "upload_dir", None) or "output/uploads"
56         ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
57         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
58         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
59         exts = [
60             e.lower().lstrip(".")
61             for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
62         ]
63         return upload_dir, max_bytes, part_bytes, exts
64
65
66     def _abort_upload(upload_id: str) -> None:
67         with _uploads_lock:
68             st = _uploads.pop(upload_id, None)
69             if st:
70                 try:
71                     os.remove(st["tmp_path"])
72                 except OSError:
73                     pass
74
75
76     def _normalize_upload_filename(filename: str) -> str:
77         """Normalize browser-provided file names before extension validation.
78
79         Some mobile/file-provider pickers can hand us visually normal names with
80 compatibility
81         characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
82 existing
83         traversal/null-byte checks in ``safe_output_filename`` as the authority after this
84 cleanup.

```

```

82     """
83     return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")
84
85
86 @router.post("/api/upload/init")
87 def upload_init(req: UploadInitRequest):
88     """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
89     upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
90     try:
91         name = safe_output_filename(_normalize_upload_filename(req.filename))
92     except ValueError as e:
93         raise HTTPException(status_code=400, detail=str(e)) from e
94     ext = os.path.splitext(name)[1].lower().lstrip(".")
95     if ext not in exts:
96         raise HTTPException(
97             status_code=400,
98             detail=f"Extension '{ext}' not allowed. Allowed: {sorted(exts)}",
99         )
100     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
101         raise HTTPException(
102             status_code=413,
103             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
104         )
105     os.makedirs(upload_dir, exist_ok=True)
106     # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
107     # effort ? an
108     # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
109     # (never blocks a
110     # legit upload); the per-part total cap in upload_part is the hard backstop either
111     # way.
112     require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
113     upload_id = uuid.uuid4().hex
114     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
115     open(tmp_path, "wb").close()
116     with _uploads_lock:
117         _uploads[upload_id] = {
118             "filename": name,
119             "tmp_path": tmp_path,
120             "parts": 0,
121             "bytes": 0,
122         }
123     return {
124         "upload_id": upload_id,
125         "max_part_mb": part_bytes // (1024 * 1024),
126         "next_part": f"/api/upload/{upload_id}/part/0",
127     }
128
129 @router.put("/api/upload/{upload_id}/part/{index}")
130 async def upload_part(upload_id: str, index: int, request: Request):
131     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
132     _, max_bytes, part_bytes, _ = _upload_cfg()
133     with _uploads_lock:
134         st = _uploads.get(upload_id)
135         if st is None:
136             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
137         if index != st["parts"]:
138             raise HTTPException(
139                 status_code=409,
140                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are sequential)",
141             )
142     received = 0
143     overflow = None

```

```

143     with open(st["tmp_path"], "ab") as fh:
144         async for chunk in request.stream():
145             received += len(chunk)
146             if received > part_bytes:
147                 overflow = (
148                     f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
149                 )
150                 break
151             if st["bytes"] + received > max_bytes:
152                 overflow = (
153                     f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
154                 )
155                 break
156             fh.write(chunk)
157     if overflow:
158         _abort_upload(upload_id)
159         raise HTTPException(
160             status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
161         )
162
163     with _uploads_lock:
164         st["parts"] += 1
165         st["bytes"] += received
166     return {
167         "upload_id": upload_id,
168         "received_parts": st["parts"],
169         "received_bytes": st["bytes"],
170     }
171
172
173     @router.post("/api/upload/{upload_id}/complete")
174     def upload_complete(upload_id: str, req: UploadCompleteRequest):
175         """Finalize: verify part count, move into place, return {path, video_id} for
176 /api/process."""
177         upload_dir, _, _, _ = _upload_cfg()
178         with _uploads_lock:
179             st = _uploads.get(upload_id)
180             if st is None:
181                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
182             if st["parts"] == 0 or st["parts"] != req.total_parts:
183                 raise HTTPException(
184                     status_code=409,
185                     detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
186                 )
187             final_path = os.path.join(upload_dir, st["filename"])
188             if os.path.exists(
189                 final_path
190             ): # never overwrite an earlier upload with the same name
191                 stem, ext = os.path.splitext(st["filename"])
192                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
193             os.replace(st["tmp_path"], final_path)
194             with _uploads_lock:
195                 _uploads.pop(upload_id, None)
196             video_id = compute_video_id(final_path)
197             return {
198                 "path": os.path.abspath(final_path),
199                 "video_id": video_id,
200                 "size_bytes": os.path.getsize(final_path),
201                 "next": "POST /api/process with this path",
202             }
203
204     @router.delete("/api/upload/{upload_id}")
205     def upload_abort(upload_id: str):
206         """Abort an in-flight upload and delete its partial file."""

```

```

207         with _uploads_lock:
208             known = upload_id in _uploads
209             if not known:
210                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
211             _abort_upload(upload_id)
212             return {"status": "aborted", "upload_id": upload_id}
213

```

7. user (2026-07-05T06:54:06.982Z)

```

1         """Pydantic models for the application configuration.
2
3         This module defines the canonical, validated, typed configuration for the
4         Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6         Unknown / typo'd config keys are rejected at load time via extra="forbid".
7         """
8
9         from __future__ import annotations
10
11         from typing import Any, List, Literal, Optional
12
13         from pydantic import BaseModel, Field, field_validator, model_validator
14
15         Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18         class ValidationRelevanceConfig(BaseModel):
19             court_green_lower: list[int] = Field(default=[35, 40, 40])
20             court_green_upper: list[int] = Field(default=[85, 255, 255])
21             court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24         class ValidationConfig(BaseModel):
25             min_resolution_width: int = 1280
26             min_resolution_height: int = 720
27             min_fps: float = 29.9
28             min_blur_threshold: float = 20.0
29             max_scene_cuts_per_minute: float = 0.5
30             pts_max_gap_seconds: float = Field(
31                 default=10.0,
32                 ge=0.0,
33                 description=(
34                     "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35                     "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36                     "a gap >10s suggests edited/missing content. Mobile phones, screen "
37                     "recordings, and other encoders may use different GOP sizes ? tune "
38                     "this threshold for your recording device."
39                 ),
40             )
41             relevance: ValidationRelevanceConfig = Field(
42                 default_factory=ValidationRelevanceConfig
43             )
44
45
46         class TrackNetConfig(BaseModel):
47             wsl_distro: str = "Ubuntu"
48             repo_dir: str = "~/models/TrackNetV4"
49             conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50             conda_env: str = "TrackNetV4"
51             weights_path: str = ""
52             queue_length: int = 5
53             stage_in_wsl: bool = True
54             wsl_stage_dir: str = "~/clips"
55             timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)

```

```

56     velocity_threshold: float = 0.02
57     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58     # decoded frame cache) are deleted automatically ? they are pure intermediates,
59     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61     keep_frames: bool = False
62     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63     # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
64     # 0 = no extra floor (the headroom guard still applies).
65     min_free_gb: float = 0.0
66
67
68     class WasbIndexConfig(BaseModel):
69         """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72         these knobs actually take effect ? previously this whole block was silently dropped
73         because nested config sections defaulted to extra='ignore'.
74         """
75
76         wsl_distro: str = "Ubuntu"
77         repo_dir: str = "~/models/WASB-SBDT"
78         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79         conda_env: str = "wasb"
80         weights_path: str = (
81             "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82         )
83         sport: str = "badminton"
84         wsl_stage_dir: str = "~/clips"
85         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86         velocity_threshold: float = 0.02
87         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
88         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91         device: str = "cuda" # auto | cuda | mps | cpu
92         python_bin: str = (
93             "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94         )
95         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98         keep_frames: bool = False
99         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100        # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101        # 0 = no extra floor (the headroom guard still applies).
102        min_free_gb: float = 0.0
103        # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104        # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105        # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106        # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107        # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).

```

```

108     # Tri-state:
109     # None      = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110     #          behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111     # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112     stream_video: Optional[bool] = None
113     # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114     # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115     # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
116     # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117     # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118     batch_size: Optional[int] = None
119     # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120     # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121     prefetch_batches: Optional[int] = None
122
123     @field_validator("batch_size")
124     @classmethod
125     def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
126         # A DataLoader batch must be >=1; reject at config time so a typo can't reach
127         # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
128         # when truthy, so 0 stays "off/parity-default").
129         if v is not None and v < 1:
130             raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
131         return v
132
133     @field_validator("prefetch_batches")
134     @classmethod
135     def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
136         # 0 = off (the default). Reject negatives: the runner passes any truthy value as
137         # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
138         # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
139         if v is not None and v < 0:
140             raise ValueError(
141                 f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
142             )
143         return v
144
145
146     class FusionConfig(BaseModel):
147         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
148
149         Every default reproduces control behaviour: the fusion segmenter persists the
shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
150
151         true ? the person-cue features, but it does NOT gate the served handoff unless a
threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
152
153         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
154
155         supplied ? off-court persons (spectators / adjacent courts) are excluded.
156         """
157
158         # Which trajectory substrate seeds the windows (shuttle stays primary).
159         substrate: Literal["wasb", "tracknet"] = "wasb"
160
161         # Windowing velocity threshold for the fusion family (the engine reads

```



```

160     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
161     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
162     velocity_threshold: float = 0.02
163     # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
164     # enabled ? so the default path never imports torch / touches the GPU.
165     cue_flags: dict[str, bool] = Field(default_factory=dict)
166     # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
167     # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
168     min_crossings: int = Field(default=0, ge=0)
169     # Served Gate B: when the person cue is on, drop windows with median in-court
players
170     # < this. 0 = OFF.
171     min_players: int = Field(default=0, ge=0)
172     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
173     court_polygon: Optional[list[list[float]]] = None
174     # Net line for the person two-sided-motion split (person features only ? the shuttle
175     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
176     net_axis: Literal["x", "y"] = "y"
177     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
178
179
180     class IndexingConfig(BaseModel):
181         default_segmenter: str = "yolo_hybrid"
182         use_gpu: bool = True
183         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
184         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
185         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
186         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
187         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
188         detector_impl: str = "auto"
189         motion_threshold: float = 0.08
190         min_segment_duration: float = 3.0
191         dead_time_merge_gap: float = 2.0
192         chunk_duration: float = 90.0
193         chunk_overlap: float = 10.0
194         detector_model: str = "fasterrcnn_resnet50_fpn"
195         detector_score_threshold: float = 0.5
196         yolo_velocity_threshold: float = 0.15
197         yolo_frame_skip: int = 3
198         yolo_tracker: str = "bytetrack.yaml"
199         yolo_prefetch_workers: int = 2
200         min_action_window_duration: float = 2.0
201         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
202         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
203         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
204         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
205         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
206         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
207         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
208         max_window_duration: float = 6.0
209         window_min_split_gap: float = (
210             0.5 # min internal gap (s) that qualifies as a split point
211         )
212         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap

```

```

213      # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
214      # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
215      # Only cuts (recall can't drop). OFF by default until measured/promoted.
216      window_hard_cap: bool = False
217      # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
218      # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
219      # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
220      # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
221      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
222      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
223      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
224      window_inactivity_end: float = 1.5
225      inactivity_rally_thresh: float = 0.20
226      inactivity_min_density: float = 0.30
227      # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
228      # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
229      # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
230      # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
231      # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
232      # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
233      # regression), |?end| 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
234      # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
235      # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
236      inactivity_temporal_density: bool = True
237      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
238      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
239      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
240      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
241      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
242      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
243      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
244      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
245      window_blob_fallback: bool = False
246      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
247      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
248      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
249      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".

```

```

250     window_blob_factor: float = 4.0
251     log_yolo_candidates: bool = True
252     store_yolo_candidates: bool = True
253     ai_handoff_padding: float = 2.0
254     ai_max_workers: int = 5
255     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
256     # chunks of a high-bitrate (4K) source blow past the provider upload cap
257     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
258     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
259     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
260     # Matches gemini_refine's --clip-scale-width default.
261     ai_chunk_scale_width: int = 1280
262     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
263     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
264     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
265     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
266     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
267     skip_ai_handoff: bool = False
268     # Optional debug knob: trim every video to the first N seconds before processing.
269     debug_duration: Optional[float] = None
270
271     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
272     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
273     fusion: FusionConfig = Field(default_factory=FusionConfig)
274
275     @field_validator("default_segmenter")
276     @classmethod
277     def segmenter_must_be_registered(cls, v: str) -> str:
278         # Lazy import: the segmenter registry imports modules that may import
279         # config, so we resolve it at validation time rather than at module load.
280         from backend.pipeline.segmenters.base import segmenter_registry
281
282         if segmenter_registry.get(v) is None:
283             available = sorted(segmenter_registry.list_available().keys())
284             raise ValueError(
285                 f"Segmenter '{v}' is not registered. Available: {'', '.join(available)}"
286             )
287         return v
288
289     @model_validator(mode="after")
290     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
291         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
292         # step makes `while t < duration: t += step` loop forever, growing memory before
any
293         # GPU work. Reject the bad config at load time instead.
294         if self.chunk_overlap >= self.chunk_duration:
295             raise ValueError(
296                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
297                 f"({self.chunk_duration}); otherwise chunking never advances."
298             )
299         return self
300
301
302     class OutputConfig(BaseModel):
303         default_highlights_name: str = "highlights.mp4"
304         db_name: str = "sports_indexer.db"
305         # Directory where the DB, highlight reels, and processed clips are written.
306         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
307         output_dir: str = "output"

```

```

308
309
310 class WatermarkConfig(BaseModel):
311     """Branding overlay burned into each highlight clip during the per-clip re-encode
312     (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
313     (under `stitching.watermark` in config.json) to turn it off. The text is fully
314     configurable. The concat stage stays stream-copy (-c copy)."""
315
316     enabled: bool = True
317     text: str = "Generated by Khelsutra.guru"
318     logo_path: str = "" # optional transparent PNG overlay; "" = no logo
319     font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family below
320     font: str = "Sans" # fontconfig family used when font_path is empty
321     font_size_ratio: float = Field(
322         default=0.035, gt=0.0, le=0.5
323     ) # text height as a fraction of frame height
324     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
325     box: bool = True # faint backing box behind the text for legibility
326     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
327     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
328         "bottom-right"
329     )
330
331
332 class StitchingConfig(BaseModel):
333     begin_padding: float = 0.5
334     end_padding: float = 0.5
335     use_cache: bool = False
336     min_shot_count: int = 1
337     # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
338     # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
339     # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
340     # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
341     # local detector over-fires and its false positives are mostly SHORT (motion blips,
342     # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
343     # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
344     min_rally_duration: float = Field(default=0.0, ge=0.0)
345     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
346
347
348 class LoggingConfig(BaseModel):
349     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
350
351     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
352     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
353     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
354     # them to WARNING; set true to restore full chatter for request-flow debugging.
355     verbose_http: bool = False
356
357
358 class SecurityConfig(BaseModel):
359     # When set, /api/process video_path must resolve under this directory (hosted/tenant
360     # mode). Default None preserves the single-user local 'any local file' workflow.
361     allowed_video_root: Optional[str] = None
362
363     # --- Chunked upload (B2, PR-2) ---
364     # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
365     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
366     # contain upload_dir or /api/process will reject the uploaded paths.
367     upload_dir: str = "output/uploads"

```

```

368 # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
369 # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
370 max_upload_mb: int = 4096
371 max_part_mb: int = 95
372 allowed_upload_extensions: List[str] = ["mp4", "mov"]
373
374 # --- M9 edge auth (Cloudflare Access, D9) ---
375 # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
376 # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
377 # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
378 # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
379 # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
380 # application's audience tag. NO secrets here ? these are public identifiers.
381 edge_auth_enabled: bool = False
382 cf_team_domain: str = (
383     "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
384 )
385 cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
386 # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
387 # backend/main.py so importing the app does no filesystem I/O; default [""].
Credentials stay
388 # OFF ? Cf-Access is a header, not a cookie.)
389
390
391 class StorageConfig(BaseModel):
392     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
393     behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
394     + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
395     Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
396     **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
397     (boto3/google auth chains supply credentials). See ``backend/storage/``. """
398
399     backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
400     # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
401     output_root: str = ""
402     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
403     # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
404     # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
405     # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
406     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
407     # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
408     # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
409     # scratch dir before processing (see ``backend.storage.acquire_local_input``).
410     input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
411     input_root: str = ""
412     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
413     # One folder per video keyed by the MD5 video_id:

```

```

<root>/[<workspace_prefix>]/<video_id>/...
414     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
415     # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
416     # the legacy flat layout is used unchanged.
417     single_folder_per_video: bool = False
418     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
419     workspace_root: str = ""
420     # Cloud namespace prefix under the root so per-video folders sit tidily beside
421     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
422     workspace_prefix: str = "workspaces"
423     local: dict[str, Any] = Field(default_factory=dict)
424     s3: dict[str, Any] = Field(default_factory=dict)
425     gcs: dict[str, Any] = Field(default_factory=dict)
426     gdrive: dict[str, Any] = Field(default_factory=dict)
427     # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
428     # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
429     gdrive_api: dict[str, Any] = Field(default_factory=dict)
430
431
432     class DeploymentConfig(BaseModel):
433         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
434         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
435
436         **Pure / additive:** the empty default profile (````) applies NO preset, so the
pipeline
437         behaves exactly as before this section existed. A profile is opt-in (``config.json``
438         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
439         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
440         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
441         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
442
443         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
444
445         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
446         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
447         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
448         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
449         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
450         # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
451         # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
452         # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
453         # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
454         # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
455         max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
456
457
458     class BillingConfig(BaseModel):
459         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
460

```

```

461     **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
462     today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
463     ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
464     version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
465     inserts a real, calibrated version and points ``pricebook_version`` at it.
466
467     ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
468     wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
469     pricebook + the recording seam; usage emission (the metering hooks) is a separate
step.
470
471     enabled: bool = False
472     pricebook_version: str = (
473         "placeholder-v0" # the seeded $0 version; owner flips to a real one
474     )
475     fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore market)
476     margin: float = 0.0 # resale markup (?0); 0 = bill at cost
477
478
479     class FlywheelConfig(BaseModel):
480         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
481         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
482         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
483         the owned model climbs toward the D11 ship floor.
484
485         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
486         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
487         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
488         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-row).
489         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
490         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
491         owner/counsel/GPU scope. Pairs with M9-consent.
492
493         enabled: bool = False
494         workspace_prefix: str = (
495             "flywheel" # per-video workspace prefix for captured artifacts
496         )
497         shadow_eval: bool = True # run the owned-vs-Gemini shadow comparison on capture
498         min_promote_n: int = 5 # promote_if_served_safe `min_n` (the served-safe gate)
499
500
501     class AppConfig(BaseModel):
502         """Root configuration object.
503
504         All runtime configuration should be accessed via an instance of this class
505         (or the loader) rather than raw dict lookups.
506
507
508         sport: Sport = "badminton"
509         ai_provider: str = "gemini"

```

```

510     # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
511     # the alias never deprecates underneath us. Eval reports record the run date
alongside.
512     ai_model: str = "gemini-flash-latest"
513
514     validation: ValidationConfig = Field(default_factory=ValidationConfig)
515     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
516     output: OutputConfig = Field(default_factory=OutputConfig)
517     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
518     security: SecurityConfig = Field(default_factory=SecurityConfig)
519     logging: LoggingConfig = Field(default_factory=LoggingConfig)
520     storage: StorageConfig = Field(default_factory=StorageConfig)
521     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
522     billing: BillingConfig = Field(default_factory=BillingConfig)
523     flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
524
525     # Reject unknown keys ? typed config migration is complete (D3).
526     model_config = {
527         "extra": "forbid",
528         "validate_assignment": True,
529     }
530
531
532     # Convenience type alias for callers that want to annotate
533     Config = AppConfig
534

```

8. assistant (2026-07-05T06:54:12.432Z)

Now let me read the freespace guard and understand how the SPA drives the upload (part size).

9. user (2026-07-05T06:54:13.455Z)

```

1     """Free-space guards for the local box (STUDIO_MEDIA_LIFECYCLE.md P2, D11).
2
3     The single-box appliance fills its disk in three peaks: a chunked **upload** (raw file
into
4     ``upload_dir``), a **fetch-to-scratch** of a bucket/Drive source (a full local copy
under the temp
5     dir), and ? the real balloon ? **WSL frame extraction**, which can materialise ~15x the
source in
6     PNGs. D11: BLOCK on a local/WSL shortfall (buckets are warn-only elsewhere ? they aren't
7     capacity-bounded from here).
8
9     ``require_free_bytes`` is the HTTP-boundary guard: it raises **HTTP 507 (Insufficient
Storage)**
10    when the filesystem holding ``path`` can't hold ``need`` bytes, so the client fails fast
instead of
11    crashing a job mid-flight. It is best-effort about *measuring* (an unreadable
``disk_usage`` never
12    blocks ? inability to measure must not deny a legitimate request), but strict once a
real shortfall
13    is seen. The WSL frame-stage guard lives on ``wslCommandMixin`` (a different filesystem,
measured via
14    ``df`` inside WSL, not ``shutil.disk_usage``) ? see ``pipeline/detectors/base.py``.
15    """
16
17    from __future__ import annotations
18
19    import logging
20    import shutil
21    from typing import Optional
22
23    from fastapi import HTTPException

```



```

24
25     logger = logging.getLogger(__name__)
26
27
28     def free_bytes(path: str) -> Optional[int]:
29         """Best-effort free bytes on the filesystem holding ``path`` (``None`` if
undeterminable).
30
31         Never raises. ``shutil.disk_usage`` wants an existing path; we walk up to the
nearest existing
32         ancestor (an ``upload_dir``/scratch target may not exist yet at guard time)."""
33         import os.path
34
35         probe = path
36         for _ in range(64): # bounded ancestor walk (defensive against a pathological path)
37             if not probe:
38                 return None
39             try:
40                 return int(shutil.disk_usage(probe).free)
41             except (OSError, ValueError):
42                 parent = os.path.dirname(probe.rstrip("/\\"))
43                 if parent == probe: # reached the root and still can't stat ? give up
44                     return None
45                 probe = parent
46         return None
47
48
49     def require_free_bytes(
50         path: str, need: Optional[int], *, stage: str = "write"
51     ) -> None:
52         """Raise HTTP 507 if ``path``'s filesystem can't hold ``need`` bytes; otherwise
return.
53
54         ``need`` of ``None`` or ``<= 0`` is a no-op (nothing to reserve). Measuring is best-
effort: an
55         undeterminable free-space read logs a warning and proceeds (we never deny on
inability to
56         measure). A confirmed shortfall raises so the caller fails fast with an actionable
507."""
57         if not need or need <= 0:
58             return
59         free = free_bytes(path)
60         if free is None:
61             logger.warning(
62                 "free-space guard (%s): could not read free space at %s ? proceeding.",
63                 stage,
64                 path,
65             )
66             return
67         if free < need:
68             need_mb = need / (1024 * 1024)
69             free_mb = free / (1024 * 1024)
70             logger.warning(
71                 "free-space guard (%s): %s needs %.0f MB but only %.0f MB free ? refusing
(507).",
72                 stage,
73                 path,
74                 need_mb,
75                 free_mb,
76             )
77             raise HTTPException(
78                 status_code=507,
79                 detail=(
80                     f"Not enough free disk space for {stage}: need ~{need_mb:.0f} MB, "
81                     f"only ~{free_mb:.0f} MB free. Free some space and retry."

```

```
82         ),
83     )
84
```

10. user (2026-07-05T06:54:17.464Z)

Already up to date.
web/src/api/client.ts
web/src/api/types.ts
web/src/api/upload.test.ts
web/src/components/IngestPanel.test.tsx
web/src/components/IngestPanel.tsx
web/src/components/upload.test.tsx
web/src/hooks/useUpload.test.ts
web/src/hooks/useUpload.ts

11. assistant (2026-07-05T06:54:20.392Z)

The key finding is already emerging. Let me examine the SPA upload client to confirm the part-chunking size it uses, and check how `require\_free\_bytes` interacts with a 2.68 GB declared size on a RAM-backed /tmp.

12. user (2026-07-05T06:54:21.401Z)

```
1      // Drives the in-UI file UPLOAD half of ingest (tracker T4): chunk a local file up to
the engine's
2      // /api/upload/* endpoints, report byte progress, then hand the returned server PATH to
the existing
3      // processing back-half (useIngestJob.submit). Alpha users have a FILE, not a gs:// URI
? this is the
4      // keystone that lets them point the tool at it from the browser.
5      //
6      // Kept i18n-FREE (returns stable IngestErrorCodes via classifyError, not prose) so it
unit-tests
7      // without a translation context ? IngestPanel maps codes to localized strings via
errorText. Abort
8      // is an AbortController: cancel/unmount/a superseding pick stops the in-flight upload
and the
9      // orchestrator best-effort DELETes the server-side partial.
10     import { useCallback, useEffect, useRef, useState } from "react";
11     import { uploadFile } from "../api/client";
12     import { classifyError, type IngestError } from "../lib/errors";
13     import { log, newCorrelationId } from "../lib/log";
14
15     export type UploadPhase =
16       | { status: "idle" }
17       | { status: "uploading"; pct: number; filename: string }
18       | { status: "error"; error: IngestError };
19
20     export function useUpload(onComplete: (path: string, filename: string) => void,
maxUploadBytes?: number) {
21       const [phase, setPhase] = useState<UploadPhase>({ status: "idle" });
22       const ctrlRef = useRef<AbortController | null>(null);
23       // Keep the latest onComplete without making start depend on it (a parent re-render
won't restart).
24       const onCompleteRef = useRef(onComplete);
25       onCompleteRef.current = onComplete;
26       // The engine's max upload size (BYTES) from GET /api/capabilities ? read through a
ref so `start`
27       // stays stable when the capability loads (same pattern as onCompleteRef). Absent/0 ?
no client guard.
28       const maxUploadBytesRef = useRef(maxUploadBytes);
29       maxUploadBytesRef.current = maxUploadBytes;
30
```

```

31     const stop = useCallback(() => {
32         ctrlRef.current?.abort();
33         ctrlRef.current = null;
34     }, []);
35
36     // Abort any in-flight upload on unmount.
37     useEffect(() => stop, [stop]);
38
39     const start = useCallback(
40         (file: File) => {
41             // A new pick supersedes any prior upload ? abort it FIRST, before the early-
return guards. If a
42             // rejected pick (0-byte / oversized) returned before stop(), an older in-flight
upload would keep
43             // running and could still call onComplete for the OLD file after we've shown an
error for the new
44             // one (processing A while the UI reports B was rejected).
45             stop();
46
47             // A 0-byte "video" is never valid ? reject before touching the network (reuse
invalidUri copy).
48             if (!file || file.size === 0) {
49                 setPhase({ status: "error", error: { code: "invalidUri" } });
50                 return;
51             }
52             // Oversized file: fail INSTANTLY (no wasted round-trip) ? /upload/init would 413
anyway. Carry
53             // the size + cap so IngestPanel can localize a "trim/chunk it first" message. No
cap ? skip the
54             // guard (the engine still fail-fasts server-side).
55             const cap = maxUploadBytesRef.current;
56             if (cap && file.size > cap) {
57                 setPhase({ status: "error", error: { code: "fileTooLarge", sizeBytes: file.size,
capBytes: cap } });
58                 return;
59             }
60
61             const ctrl = new AbortController();
62             ctrlRef.current = ctrl;
63             const cid = newCorrelationId();
64             setPhase({ status: "uploading", pct: 0, filename: file.name });
65             log("info", "upload.submit_start", { cid, filename: file.name, size: file.size });
66
67             void (async () => {
68                 try {
69                     const res = await uploadFile(file, {
70                         cid,
71                         signal: ctrl.signal,
72                         onProgress: ({ sentBytes, totalBytes }) => {
73                             if (ctrl.signal.aborted) return;
74                             const pct = totalBytes > 0 ? Math.min(100, Math.round((sentBytes /
totalBytes) * 100)) : 0;
75                             setPhase({ status: "uploading", pct, filename: file.name });
76                         },
77                     });
78                     if (ctrl.signal.aborted) return;
79                     setPhase({ status: "idle" }); // hand off ? the engine processing phase takes
over
80                     onCompleteRef.current(res.path, file.name);
81                 } catch (err) {
82                     // User-cancelled (abort) is silent, not an error.
83                     if (ctrl.signal.aborted || (err instanceof DOMException && err.name ===
"AbortError")) return;
84                     setPhase({ status: "error", error: classifyError(err) });
85                     log("error", "upload.failed", { cid, filename: file.name, error: String(err)

```

```

});
86         }
87     });
88     },
89     [stop],
90 );
91
92     const cancel = useCallback(() => {
93         stop();
94         setPhase({ status: "idle" });
95         log("info", "upload.cancel", {});
96     }, [stop]);
97
98     const reset = useCallback(() => {
99         stop();
100        setPhase({ status: "idle" });
101    }, [stop]);
102
103    const busy = phase.status === "uploading";
104    return { phase, start, cancel, reset, busy };
105 }
106

```

13. user (2026-07-05T06:54:23.321Z)

```

C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:244:// A browser uploads
a LOCAL file in sequential parts ? max_part_mb (free-tier edge proxies cap
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:318: // Honor the
engine's max_part_mb, but never let a missing/NaN value collapse the math to NaN (which
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:320: const partMb =
Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ? init.max_part_mb : 95;
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:321: const partBytes =
partMb * 1024 * 1024;
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:323: log("info",
"upload.start", { cid, filename: file.name, size: file.size, parts: totalParts, part_mb:
init.max_part_mb });
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\upload.test.ts:8:const MB = 1024 *
1024;
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\upload.test.ts:21: const calls = {
init: 0, parts: [] as number[], partSizes: [] as number[], complete: null as number | null,
deletes: 0 };
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\upload.test.ts:28: return ok({
upload_id: "up1", max_part_mb: 1, next_part: "/api/upload/up1/part/0" });
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\upload.test.ts:36:
calls.partSizes.push(size);
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\upload.test.ts:56: it("slices into
sequential parts ? max_part_mb, completes, and returns {path, video_id}", async () => {
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\upload.test.ts:64:
expect(calls.partSizes).toEqual([1 * MB, 1 * MB, 0.5 * MB]); // last part is the remainder
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\IngestPanel.test.tsx:256:
if (u === "/api/upload/init") return httpOk(200, { upload_id: "up1", max_part_mb: 95, next_part:
"/api/upload/up1/part/0" });
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\types.ts:158: * ? max_part_mb,
then `complete` returns a server-host `path` to feed straight into POST /api/process. */
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\types.ts:162: max_part_mb: number;
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\types.ts:163: next_part: string;
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\IngestPanel.test.tsx:82:
typeof c.max_upload_mb === "number" && c.max_upload_mb > 0 ? c.max_upload_mb * 1024 * 1024 :
undefined,
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.test.ts:12:const MB =
1024 * 1024;
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.test.ts:22: if (u
=== "/api/upload/init") return ok({ upload_id: "up1", max_part_mb: 1, next_part:
"/api/upload/up1/part/0" });
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\upload.test.tsx:20:const MB

```

```

= 1024 * 1024;
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\upload.test.tsx:30:   if (u
=== "/api/upload/init") return ok({ upload_id: "up1", max_part_mb: 1, next_part:
"/api/upload/up1/part/0" });

```

14. assistant (2026-07-05T06:54:28.523Z)

Let me read the client.ts upload code precisely and check how init sends total_size_bytes, plus the capabilities endpoint that reports the fields.

15. user (2026-07-05T06:54:30.050Z)

```

240     return cfg.stitching?.min_shot_count;
241   }
242
243   // ?? chunked upload (backend/api/uploads.py) ?????????????????????????????????
244   // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
245   // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
246
247   /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[backend/api/uploads.py]. */
248   export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
249     return request<UploadInitResponse>("/upload/init", {
250       method: "POST",
251       body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
252       cid,
253     });
254   }
255
256   /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[backend/api/uploads.py]. */
257   export async function uploadPart(
258     uploadId: string,
259     index: number,
260     chunk: Blob,
261     cid?: string,
262     signal?: AbortSignal,
263   ): Promise<UploadPartResponse> {
264     const c = cid ?? newCorrelationId();
265     const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
266     log("info", "api.request", { cid: c, method: "PUT", url });
267     let res: Response;
268     try {
269       res = await fetch(url, {
270         method: "PUT",
271         body: chunk,
272         headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
273         signal,
274       });
275     } catch (err) {
276       log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
});
277       throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
278     }
279     if (!res.ok) {
280       const body = await res.text().catch(() => "");
281       log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
282       throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
283     }
284     log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });

```

```

285     return (await res.json()) as UploadPartResponse;
286 }
287
288 /** Finalize: returns {path, video_id} ? feed `path` to processVideo()
[backend/api/uploads.py]. */
289 export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
290     return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
291         method: "POST",
292         body: JSON.stringify({ total_parts: totalParts }),
293         cid,
294     });
295 }
296
297 /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[backend/api/uploads.py]. */
298 export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
299     await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });
300 }
301
302 export interface UploadProgress {
303     sentBytes: number;
304     totalBytes: number;
305 }
306
307 /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
308 * On ANY failure or abort it best-effort DELETES the server-side partial, then
rethrows so the
309 * caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}.
310 * Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
recovery is a
311 * fresh init (re-pick the file), which the UI's retry does. */
312 export async function uploadFile(
313     file: File,
314     opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
= {},
315 ): Promise<UploadCompleteResponse> {
316     const cid = opts.cid ?? newCorrelationId();
317     const init = await uploadInit(file.name, file.size, cid);
318     // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
to NaN (which
319     // would silently upload zero parts) ? fall back to the engine's 95 MB default.
320     const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
321     const partBytes = partMb * 1024 * 1024;
322     const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
323     log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
324     let sent = 0;
325     try {
326         for (let i = 0; i < totalParts; i++) {
327             if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
328             const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
329             await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
330             sent += chunk.size;
331             opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
332         }
333         const done = await uploadComplete(init.upload_id, totalParts, cid);
334         log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
335         return done;

```

```

336     } catch (err) {
337         log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
338         await uploadAbort(init.upload_id, cid).catch(() => {});
339         throw err;
340     }
341 }
342

```

16. user (2026-07-05T06:54:32.063Z)

```

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:83:from
backend.storage.capabilities import (
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:84:
storage_capabilities, # P3: secret-free "which media can this box use" report
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:90:from
backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:359:@app.get("/api/capabilities")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:360:def
capabilities(request: Request):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:402:
"storage": storage_capabilities(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:569:
require_free_bytes(cache_dir, need, stage="source-cache")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:726:
from backend.storage.capabilities import _gdrive_mount_root
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:798:
require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:842:
require_free_bytes(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:952:
require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\uploads.py:3:Browser uploads arrive in sequential parts ?
security.max_part_mb (free-tier edge
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\uploads.py:28:from backend.utils.freespace import require_free_bytes #
HTTP-507 disk guard (P2, D11)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\uploads.py:57:
max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\uploads.py:58:
part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\uploads.py:109:
require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\api\uploads.py:122:
"max_part_mb": part_bytes // (1024 * 1024),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:370:
max_upload_mb: int = 4096
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\config\models.py:371:
max_part_mb: int = 95
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\capabilities.py:3:`GET /api/capabilities` surfaces this so the Studio
medium picker (P5) renders only the backends
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\capabilities.py:50:         return int(shutil.disk_usage(probe).free
/ (1024*3))
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\capabilities.py:104:     except Exception: # noqa: BLE001 - detection is
best-effort; never fail capabilities
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\capabilities.py:120:def storage_capabilities(
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\capabilities.py:123:     """Build the ``storage`` capabilities block (see
module docstring). ``local_free_path`` is the

```

```

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\freespace.py:9: `require_free_bytes` is the HTTP-boundary guard: it raises **HTTP 507 (Insufficient Storage)**
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\freespace.py:11: crashing a job mid-flight. It is best-effort about *measuring* (an unreadable `disk_usage` never
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\freespace.py:14: `df` inside WSL, not `shutil.disk_usage`) ? see `pipeline/detectors/base.py`.
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\freespace.py:28: def free_bytes(path: str) -> Optional[int]:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\freespace.py:31:     Never raises. `shutil.disk_usage` wants an existing path; we walk up to the nearest existing
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\freespace.py:40:         return int(shutil.disk_usage(probe).free)
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\freespace.py:49: def require_free_bytes(
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\freespace.py:59:     free = free_bytes(path)
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\orchestration.py:165: Surfaced in `/api/capabilities` so the SPA only offers the cloud toggle when it would work.""
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\ui\assets\index-Cqr7zJeJ.js:60: [Omitted long matching line]
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\ui\assets\index-Cqr7zJeJ.js:62: [Omitted long matching line]

```

17. assistant (2026-07-05T06:54:44.464Z)

Now I have the full picture. Let me verify the capabilities endpoint to confirm max_upload_mb/max_part_mb are exposed in current master (and thus absent in the deployed old build), and confirm the middleware doesn't impose a global body limit.

18. user (2026-07-05T06:54:45.480Z)

```

359     @app.get("/api/capabilities")
360     def capabilities(request: Request):
361         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
362         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
363         value. Best-effort + never raises (the wizard degrades gracefully)."""
364         cfg = request.app.state.config
365         indexing = getattr(cfg, "indexing", None)
366         deployment = getattr(cfg, "deployment", None)
367         ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
368         ffprobe = shutil.which(ffprobe_bin())
369         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
370         wasb = getattr(indexing, "wasb", None)
371         weights_path = (
372             os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
373         )
374         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
375         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
376         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck
is the
377         # authority on whether GPU inference actually works; this is just the wizard's
first-order
signal.
378         nvidia_smi = shutil.which("nvidia-smi")
379         return {
380             "api_version": API_VERSION,
381             "engine_version": _engine_version(),
382             "segmenters": sorted(segmenter_registry.list_available().keys()),

```



```

383     "default_segmenter": getattr(indexing, "default_segmenter", None),
384     "ffmpeg": {
385         "ffmpeg": bool(ffmpeg),
386         "ffprobe": bool(ffprobe),
387         "ffmpeg_path": ffmpeg,
388         "ffprobe_path": ffprobe,
389     },
390     "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
391     "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
392     "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
393     "deployment": {
394         "profile": getattr(deployment, "profile", "") or "",
395         "compute_target": getattr(deployment, "compute_target", "") or "",
396         # item 3: can this server satisfy a per-request compute="cloud" dispatch?
397         # the "run in the cloud" toggle only when true (else it's local-only).
398         "cloud_available": _cloud_available(cfg),
399     },
400     # P3: which storage media THIS box can store INTO (local always; s3/gcs if
401     # gdrive if a mount resolves). The P5 medium picker renders only usable
402     # SDK+config;
403     # gdrive if a mount resolves). The P5 medium picker renders only usable
404     # backends. No secrets.
405     "storage": storage_capabilities(
406         getattr(cfg, "storage", None), local_free_path=output_base(cfg)
407     ),
408 }
409
410 def _friendly_video_name(filepath: str) -> str:
411     """A human name for a video from its stored path/URI ? the filename stem
412     (``~/match_2026.mp4`` ?
413     ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
414     (rsplit on ``/``).
415     Falls back to the whole ``filepath`` if there's no basename."""
416     base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
417     stem = os.path.splitext(base)[0]
418     return stem or base or str(filepath or "")
419
420 def _video_dims(validation_results: Any) -> "tuple[Optional[float], Optional[str]]":
421     """Pull ``(fps, resolution)`` from the persisted validation results, or ``(None,
422     None)``. The

```

19. user (2026-07-05T06:54:47.467Z)

<tool_use_error>InputValidationError: Grep failed due to the following issue:
An unexpected parameter `pattter` was provided</tool_use_error>

20. assistant (2026-07-05T06:54:55.300Z)

Now let me confirm whether current master's capabilities exposes max_upload_mb/max_part_mb (the SHARED CONTEXT says it does, added ~#153). The endpoint I read does NOT include them in the top-level return, but IngestPanel.tsx reads `c.max\_upload\_mb`. Let me check where the SPA reads it from and whether it's in `security` sub-block or storage.

21. user (2026-07-05T06:54:57.258Z)

```

70     const [storeError, setStoreError] = useState<IngestError | null>(null);
71     // The engine's max upload size (BYTES), so an oversized file fails INSTANTLY client-
72     side instead of
73     // uploading into a 413. Absent ? undefined ? no guard (the engine still fail-fasts).
74     const [maxUploadBytes, setMaxUploadBytes] = useState<number | undefined>(undefined);
75     useEffect(() => {
76         let alive = true;

```

```

76     getCapabilities()
77     .then((c) => {
78         if (!alive) return;
79         setCloudAvailable(c.deployment?.cloud_available === true);
80         setBackends(c.storage?.backends ?? []);
81         setMaxUploadBytes(
82             typeof c.max_upload_mb === "number" && c.max_upload_mb > 0 ? c.max_upload_mb *
1024 * 1024 : undefined,
83         );
84     })
85     .catch((err) => log("warn", "capabilities.unavailable", { error: String(err) }));
86     return () => {
87         alive = false;
88     };
89 }, []);
90 // Only send a choice when the picker is actually shown; otherwise omit it (? server
default).
91 const sendCompute = cloudAvailable ? compute : undefined;
92 // CLOUD selected but the cost note not yet accepted ? block submit (don't silently
run local, and
93 // don't dispatch a paid job without consent).
94 const cloudBlocked = cloudAvailable && compute === "cloud" && !cloudAck;

```

22. user (2026-07-05T06:54:58.427Z)

No matches found

23. assistant (2026-07-05T06:55:04.371Z)

The SPA reads `c.max\_upload\_mb` (top-level), but the current-master capabilities endpoint I read does NOT return it. Let me confirm this is not present anywhere in main.py's capabilities return, and check git history to understand the state. The SHARED CONTEXT says current master DOES expose them (added ~#153). Let me search the whole backend once more and check git log.

24. user (2026-07-05T06:55:06.895Z)

```

64fa4a2 docs(a13): alpha launch review ? conditional-GO recommendation; GO/NO-GO to owner (#478)
122f174 docs(runlog): A1 post-merge e2e ? CUJ green on the final image; flip A1 DONE (#477)
3c97117 fix(storage): GCS keyless signing mints a cloud-platform-scoped token for signBlob
(closes #475) (#476)
195932f deploy(cloudrun): commit the control-plane service-manifest generator (closes #473)
(#474)
3b650df deploy(cloudrun): bake the auth extra (PyJWT) into the base image (M9-ready rebuilds)
(#470)
---GREP max_upload across backend---
backend/api/uploads.py:3:Browser uploads arrive in sequential parts ? security.max_part_mb
(free-tier edge
backend/api/uploads.py:57:     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) *
1024 * 1024
backend/api/uploads.py:58:     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 *
1024
backend/api/uploads.py:122:         "max_part_mb": part_bytes // (1024 * 1024),
backend/config/models.py:370:     max_upload_mb: int = 4096
backend/config/models.py:371:     max_part_mb: int = 95

```

25. assistant (2026-07-05T06:55:20.965Z)

Interesting and important: current master's `/api/capabilities` does NOT expose `max\_upload\_mb` or `max\_part\_mb` at all. The SPA reads `c.max\_upload\_mb` but the engine never returns it, so `maxUploadBytes` is always `undefined` ? the client-side fast-fail guard is inert. This means the client never pre-rejects, and the browser proceeds to chunk and upload.

Now I have everything needed to trace the exact 413. Let me do the arithmetic precisely.

- SPA slices into parts of `max\_part\_mb` MB. From `/upload/init`, deployed CP returns `max\_part\_mb`. Default is 95 MB.
- 2749 MB / 95 MB = 29 parts (28 full + remainder).
- Each part = 95 MB ? ****under**** Cloudflare's 100 MB edge body limit. So (d) is not triggered by the default 95 MB part size.
- Per-part cap in `upload\_part`: 95 MB parts vs `part\_bytes` = 95 MB ? not exceeded.
- Whole-file cap: 2749 MB < 4096 MB ? `/upload/init` whole-file check passes.
- ****The `require\_free\_bytes(upload\_dir, 2749MB, stage="upload")` at line 109**** ? `/tmp` is RAM-backed on a 4Gi Cloud Run instance. `shutil.disk\_usage("/tmp/...")` on a RAM-backed tmpfs reports the tmpfs size (memory-backed), which on a 4Gi instance is well under 2.68 GB of free space once the runtime is resident. This raises ****HTTP 507****, not 413.

Wait ? the issue says 413. Let me reconsider carefully. The free-space guard raises 507, not 413. So the observed 413 must come from a different path. Let me re-examine which path yields 413 for the DEPLOYED OLD build, since the deployed build is engine 0.1.0 and predates max_upload_mb in capabilities. Let me check the deployed old build's likely `/upload/init` behavior and whether `total\_size\_bytes` is sent. Let me verify the git history of uploads.py to see if the old build had the free-space guard and the whole-file check.

26. user (2026-07-05T06:55:23.504Z)

```
=== uploads.py history ===
f0814d5 Normalize upload filenames before extension checks
4f148d1 fix: config-correctness + typing fixes (audit R1-19, R1-20, R1-27) (#422)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
b6214e5 chore: ruff format the whole repo (259 files) (#387)
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router (#228)
=== freespace.py history ===
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
```

27. assistant (2026-07-05T06:55:33.580Z)

The free-space guard landed in #407. The deployed old build reports engine_version 0.1.0 and has NO max_upload_mb in capabilities. I need to determine whether the deployed build predates #407 (free-space guard) or includes it. Let me check what `\_engine\_version()` returns and whether there's a tag/version tied to it, and check the CORS/edge context. The critical distinction is whether the deployed build 413s at the whole-file `/upload/init` check, at a per-part check, or at the Cloudflare edge.

Let me reconsider the 413 more carefully. The most likely mechanism for a genuine ****413**** (not 507) on the deployed build. Let me check the deployed config: it sets `input\_root` to GCS but the `security` block does NOT override `max\_upload\_mb` or `max\_part\_mb`. So the deployed build uses its own code defaults. Let me verify what defaults the OLD build had ? check if max_part_mb default was ever different, and whether Cloudflare is in front.

28. user (2026-07-05T06:55:37.858Z)

```
=== old models.py security defaults (at e67add1, when uploads island was extracted) ===
256:class SecurityConfig(BaseModel):
268:    max_upload_mb: int = 4096
269:    max_part_mb: int = 95
=== even older: first appearance of max_part_mb ===
5eaba74 feat(api): chunked upload + highlight download (B2+B3) - static UI compile fix +
download link (#99)
=== _engine_version source ===
backend/main.py:338:API_VERSION = 1
backend/main.py:341:def _engine_version() -> str:
backend/main.py:353:    "engine_version": _engine_version(),
backend/main.py:381:    "engine_version": _engine_version(),
```

29. user (2026-07-05T06:55:40.955Z)

```
338    API_VERSION = 1
```

```

339
340
341     def _engine_version() -> str:
342         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
343         try:
344             return _pkg_version("badminton-highlight-indexer")
345         except PackageNotFoundError:
346             return "unknown"
347
348
349     @app.get("/api/version")
350     def api_version():
351         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
352         return {
353             "engine_version": _engine_version(),
354             "api_version": API_VERSION,
355             "ui": _bundled_ui_version(),
356         }
357

```

30. user (2026-07-05T06:55:46.354Z)

```

=== max_part_mb default at #99 (first introduction) ===
143:     max_upload_mb: int = 4096
144:     max_part_mb: int = 95
=== does old init send/check total_size_bytes + free-space? show uploads at #228 pre-freespace
===
34:     total_size_bytes: Optional[int] = None
48:     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
49:     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
51:     return upload_dir, max_bytes, part_bytes, exts
67:     upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
76:     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
77:         raise HTTPException(status_code=413,
78:                             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload
limit.")
85:     return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
92:     _, max_bytes, part_bytes, _ = _upload_cfg()
106:         if received > part_bytes:
107:             overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
109:         if st["bytes"] + received > max_bytes:
110:             overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
115:         raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-init to
retry.")

```

31. assistant (2026-07-05T06:55:55.228Z)

This is decisive. The old build (pre-#407, engine 0.1.0) had the **identical** upload logic minus the free-space guard: whole-file cap = 4096 MB, per-part cap = 95 MB, both 413. Defaults have never changed (95 MB part, 4096 MB whole-file since #99).

So on the deployed old build there is **no 507 free-space guard** ? the guard only landed in #407. That means the deployed build's 413 cannot be the free-space guard (which is a 507 anyway, and absent from the old build). The 413 on the deployed old build must be either the whole-file cap (but 2.68 GB < 4 GB, so no) or the per-part cap ? OR it's Cloudflare's edge limit surfacing as 413.

Let me nail the Cloudflare edge question. Cloudflare's default request-body limit on the Free/Pro plan is 100 MB. The SPA slices at `max\_part\_mb` = 95 MB, which is under 100 MB ? so parts should pass the edge. But the raw PUT body has HTTP overhead. Let me check whether the deployed CP actually sits behind Cloudflare, and reconsider whether the SPA could be sending

larger parts. Let me check the SPA's built bundle that's actually deployed and the request wrapper's handling of 413 vs the base URL.

32. user (2026-07-05T06:55:56.742Z)

```
1      // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
single-origin
2      // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3      // Every call carries a correlation id (header + logs) so a journey is traceable end to
end.
4      import { log, newCorrelationId } from "../lib/log";
5      import type {
6          RallySegment,
7          QueryResponse,
8          CompileResult,
9          EngineConfig,
10         EngineCapabilities,
11         ProcessResponse,
12         JobStatus,
13         UploadInitResponse,
14         UploadPartResponse,
15         UploadCompleteResponse,
16         VideoSummary,
17         VideosResponse,
18         AnnotationsResponse,
19         AssetResponse,
20     } from "../types";
21
22     /** Resolve the engine API base. Default same-origin `/api`. For a SPLIT deploy ? SPA
on
23     * Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md §2) ?
24     * set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
25     * (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
26     * Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
so
27     * `` `${base}/process` `` never double-slashes. */
28     export function apiBase(): string {
29         return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
30     }
31
32     const BASE = apiBase();
33
34     export class ApiError extends Error {
35         readonly cid?: string;
36         readonly status?: number;
37         readonly body?: string;
38         constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
39             super(message);
40             this.name = "ApiError";
41             this.cid = opts.cid;
42             this.status = opts.status;
43             this.body = opts.body;
44             if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
45         }
46     }
47
48     interface RequestOpts {
49         method?: string;
50         body?: string;
51         headers?: Record<string, string>;
```

```

52      /** Correlation id to thread through this call; auto-generated if absent. */
53      cid?: string;
54    }
55
56    async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
57      const cid = opts.cid ?? newCorrelationId();
58      const url = `${BASE}${path}`;
59      const method = opts.method ?? "GET";
60      log("info", "api.request", { cid, method, url });

```

33. user (2026-07-05T06:55:58.400Z)

```

C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:247:/** Start a chunked
upload. 400 = bad extension; 413 = over the size limit [backend/api/uploads.py]. */
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:309: * caller can
classify it (the engine's 400/413 detail survives). Returns {path, video_id}.
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\client.ts:310: * Note: a per-part
413 (over the per-part limit) aborts the upload server-side ? recovery is a
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\types.ts:103: * /upload/init
already 413s an oversized file BEFORE any part transfers; the SPA consumes this to
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\upload.test.ts:33: if
(opts.partFailsAt === idx) return httpErr(413, "Part exceeds the per-part limit");
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\api\upload.test.ts:81: await
expect(uploadFile(videoFile(2.5 * MB))).rejects.toMatchObject({ name: "ApiError", status: 413
});
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\lib\errors.ts:33: * `fileTooLarge` is
the client-side upload-size guard (rejected pre-network, carries size + cap). */
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\lib\errors.ts:34:export type
IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout" | "fileTooLarge" |
"engine";
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\lib\errors.ts:40: /** Present for
`fileTooLarge` ? the picked file's size and the engine's cap, both in BYTES, so the
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.ts:52: //
Oversized file: fail INSTANTLY (no wasted round-trip) ? /upload/init would 413 anyway. Carry
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.ts:57:
setPhase({ status: "error", error: { code: "fileTooLarge", sizeBytes: file.size, capBytes: cap }
});
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\lib\errorText.test.ts:40:
it("fileTooLarge shows the file size + the cap and prompts a trim (localized, never a raw key)",
() => {
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\lib\errorText.test.ts:41: const s =
errorText(i18n.t, { code: "fileTooLarge", sizeBytes: 6 * 1024 ** 3, capBytes: 4 * 1024 ** 3 });
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\lib\errorText.test.ts:48:
it("fileTooLarge with no size/cap degrades to a real string (defensive, never NaN or a raw
key)", () => {
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\lib\errorText.test.ts:49: const s =
errorText(i18n.t, { code: "fileTooLarge" });
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\lib\errorText.ts:17: if (e.code ===
"fileTooLarge") {
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\lib\errorText.ts:19: return
t("errors.fileTooLarge", { size, cap });
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.test.ts:25: if
(opts.partFailsAt === Number(part[1])) return httpErr(413, "Part exceeds the per-part limit");
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.test.ts:68: error:
{ code: "fileTooLarge", sizeBytes: 5 * 1024 ** 3, capBytes: cap },
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.test.ts:85: // in-
flight upload, not leave it running to completion behind the fileTooLarge error.
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.test.ts:91: error:
{ code: "fileTooLarge", sizeBytes: 5 * 1024 ** 3, capBytes: cap },
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.test.ts:95: //
clobber the fileTooLarge error.
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.test.ts:100:
expect(result.current.phase).toMatchObject({ status: "error", error: { code: "fileTooLarge" }
});
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\hooks\useUpload.test.ts:130: // a

```

```

413 with a {detail} body classifies as an engine error (its message survives)
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\IngestPanel.tsx:72:  //
uploading into a 413. Absent ? undefined ? no guard (the engine still fail-fasts).
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\IngestPanel.tsx:456:
{errorState?.code === "fileTooLarge" && (
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\IngestPanel.tsx:459:
{t("errors.fileTooLargeHelp.summary")})
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\IngestPanel.tsx:463:
<Trans i18nKey="errors.fileTooLargeHelp.phone" components={{ b: <strong /> }} />
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\IngestPanel.tsx:466:
<Trans i18nKey="errors.fileTooLargeHelp.computer" components={{ b: <strong /> }} />
C:\Users\avidu\Projects\khelsutra-guru\khelsutra\web\src\components\IngestPanel.tsx:469:
<Trans i18nKey="errors.fileTooLargeHelp.advanced" components={{ b: <strong /> }} />

```

34. assistant (2026-07-05T06:56:12.263Z)

Now I have the complete picture. The key insight is the deployment topology. The SPA uses same-origin `/api` unless `VITE\_API\_BASE` is set. The SHARED CONTEXT says the CP is a Cloud Run service serving the SPA + engine behind one origin (single-origin appliance). The question is whether Cloudflare sits in front.

The critical realization: **Cloud Run itself has a 32 MB request limit for the HTTP/1 request path** when NOT using streaming, but more importantly ? let me reconsider. The parts are 95 MB. The observed 413 with a 95 MB part strongly implicates an **edge/proxy body cap below 95 MB**, OR Cloud Run's own limit.

Actually ? Cloud Run's request body limit is **32 MB** for HTTP/1 by default and effectively unlimited only for streaming/HTTP2. But FastAPI's `request.stream()` IS streaming. The most decisive edge cap here: Cloudflare Free/Pro = 100 MB (95 MB parts pass). But if the CP is fronted by Cloudflare Access with the standard free plan, the 100 MB is the cap and 95 MB parts fit.

Let me not over-speculate on infra I can't see. Let me re-read the exact 413 arithmetic and confirm the part count and the fact that `total\_size\_bytes` IS sent (line 251 confirms it). The engine-side 413 paths are exhaustively: (a) whole-file at init, (b) per-part / running-total at upload_part. Let me verify there is no ASGI/uvicorn/starlette global body limit and confirm the deployed config doesn't shrink max_part_mb. Let me check the deploy manifest / preset for cloud-serving.

35. user (2026-07-05T06:56:13.265Z)

```

1      """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3      A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4      config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5      knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7      Precedence (lowest ? highest), realised by the loader::
8
9          built-in model defaults < profile preset < config.json < env < worker --set
10
11     This module is pure data + pure functions ? no IO, and (critically) no torch
import at
12     module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13     (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14     profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15     today). Auto-detect only ever suggests a profile ? it never auto-applies one (D15:
cloud
16     spend is never entered silently).
17     """
18
19     from __future__ import annotations
20
21     import copy

```

```

22     import os
23     from typing import Any, Mapping, Optional
24
25     # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26     # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27     PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28     COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30     # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31     # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32     COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33         "truly-local": "local_gpu",
34         "cloud-serving": "cloud_run",
35         "cpu-mock": "cpu_mock",
36     }
37
38     # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39     # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40     # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41     # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
52             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53             # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54             # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
55             # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56             # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57             "indexing": {
58                 "detector_impl": "native",
59                 "skip_ai_handoff": False,
60                 "wasb": {"batch_size": 64, "prefetch_batches": 4},
61             },
62             "storage": {"backend": "gcs"},
63         },
64         "cpu-mock": {
65             "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
66             # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
67             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
68             "storage": {"backend": "local"},
69         },
70     }
71
72
73     def is_known_profile(profile: str) -> bool:
74         """True for a non-empty, recognised profile name."""
75         return bool(profile) and profile in PROFILE_PRESETS

```



```

76
77
78 def preset_for(profile: str) -> dict[str, Any]:
79     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
80     module-level table), or ``{}`` for ``""`` / an unknown profile."""
81     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
82
83
84 def _truthy(val: Optional[str]) -> bool:
85     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
86     if val is None:
87         return False
88     return val.strip().lower() in {"1", "true", "yes", "on"}
89
90
91 def _cuda_available() -> bool:
92     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
93     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
94     try:
95         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
96         # and optional). Only reached when a caller opts into probe_cuda.
97         import torch
98
99         return bool(torch.cuda.is_available())
100     except Exception:
101         return False
102
103
104 def probe_cuda_available() -> bool:
105     """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
106
107     Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
108     that *wants* the GPU heads-up can pass the result in without the startup module
importing
109     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
110     (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
111     treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
112     error that could false-fail."""
113     return _cuda_available()
114
115
116 def suggest_profile(
117     env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
118 ) -> str:
119     """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
120
121     Signals, in priority order:
122
123     - ``K_SERVICE`` set (Cloud Run) ? ``"cloud-serving"``
124     - ``CI`` truthy ? ``"cpu-mock"``
125     - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
126     - otherwise ? ``""`` (no strong suggestion)
127
128     ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
129     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes

```

```

130         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
131         """
132         e = os.environ if env is None else env
133         if e.get("K_SERVICE"):
134             return "cloud-serving"
135         if _truthy(e.get("CI")):
136             return "cpu-mock"
137         if probe_cuda and _cuda_available():
138             return "truly-local"
139         return ""
140

```

36. assistant (2026-07-05T06:56:28.886Z)

Confirmed: the `cloud-serving` preset does NOT touch `security.max\_upload\_mb` or `security.max\_part\_mb`. And the deployed config.json (from SHARED CONTEXT) doesn't override them either. So the deployed CP uses the code defaults: ****95 MB per part, 4096 MB whole-file****.

Now the decisive question is the Cloud Run / edge body cap, since the engine-side checks (a) and (b) both pass for a 95 MB part and a 2.68 GB whole-file. Let me verify one thing that could change the arithmetic: whether the SPA bundle actually deployed (the old build) chunked at a DIFFERENT part size, or whether the old capabilities returned max_part_mb correctly. But the old CP's `/upload/init` returns `max\_part\_mb` from the same code default (95). The SPA slices at 95 MB.

The key infra fact: ****Cloud Run has a hard 32 MB request body limit for non-streamed (buffered) requests, but the streaming path lifts it ? however, the request must be sent with a proper streaming/chunked transfer.**** A browser `fetch` with a `Blob` body sets a fixed `Content-Length` (95 MB), which Cloud Run's HTTP/1 front end handles. Cloud Run's documented request size limit is 32 MiB ****only when using the HTTP/1 server without HTTP/2****; with HTTP/2 enabled it's 32 MB for the response but requests can stream. Actually, the authoritative Cloud Run limit: ****request body max is 32 MiB when NOT streaming; streaming has no fixed limit.**** A single 95 MB PUT with a fixed Content-Length is a non-streamed request from the ingress's perspective ? ****413 Payload Too Large** from the Cloud Run front end / Google Front End**, which the app never sees.

This is the strongest candidate: a 95 MB part exceeds Cloud Run's 32 MiB request limit. Let me verify there's no HTTP/2 or streaming-upload config in the deploy manifest.

37. user (2026-07-05T06:56:32.954Z)

```

=== find deploy manifest / cloud run service generator ===
cloudrun
digitalocean
gcp
./git/logs/refs/heads/fix/cloudrun-dispatch-client
./git/logs/refs/remotes/origin/feat/cloudrun-service-manifest
./git/logs/refs/remotes/origin/fix/cloudrun-dispatch-client
./git/refs/heads/fix/cloudrun-dispatch-client
./git/refs/remotes/origin/feat/cloudrun-service-manifest
./git/refs/remotes/origin/fix/cloudrun-dispatch-client
./git/worktrees/wt-cloudrun-cancel
./git/worktrees/wt-cloudrun-dispatch
./artifacts/cloudrun_e2e_validation
./artifacts/cloudrun_e2e_validation/cloudrun_e2e_reel.mp4
./artifacts/cloudrun_e2e_validation/halfA_cloudrun_execution.log
./artifacts/cloudrun_lightup
./backend/compute/cloud_run.py
./backend/compute/__pycache__/cloud_run.cpython-313.pyc
./backend/reporting/spec.yaml
./deploy/cloudrun
./deploy/cloudrun/cloudbuild.yaml
./deploy/gcp/nightly_regression/cloudbuild.launcher.yaml

```

38. user (2026-07-05T06:56:40.464Z)

```
cloudbuild.yaml
Dockerfile
gen_service_yaml.py
README.md
=== grep for use_http2 / --use-http2 / timeout / cpu / memory / concurrency in cloudrun deploy
===
deploy/cloudrun/gen_service_yaml.py:151:    p.add_argument("--cpu", type=int, default=2)
deploy/cloudrun/gen_service_yaml.py:152:    p.add_argument("--memory", default="4Gi")
deploy/cloudrun/README.md:35:  --cpu 8 --memory 32Gi \
deploy/cloudrun/README.md:47:  so size `--memory` for the largest expected upload (a 5.8 GB
GoPro file ? 6 GiB of tmpfs).
deploy/cloudrun/README.md:87:  --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
```

39. assistant (2026-07-05T06:56:43.361Z)

This is highly relevant. Let me read the `gen_service_yaml.py` and the `README` section about `tmpfs/memory` sizing ? this directly addresses the RAM-backed `/tmp` free-space concern and whether HTTP/2 is enabled.

40. user (2026-07-05T06:56:43.742Z)

```
1      """Generate the control-plane Cloud Run *service manifest* (COMPUTE_DECOUPLED_SERVING
M8/M9, #473).
2
3      The A1 alpha deploy drove `rally-control-plane` from a hand-generated YAML that lived in
a
4      session scratchpad ? this commits the generator so the deploy is reproducible from a
clean
5      clone. It emits a Knative service manifest for::
6
7      python deploy/cloudrun/gen_service_yaml.py --edge-auth off -o service.yaml
8      gcloud run services replace service.yaml --region asia-southeast1
9
10     Why a MANIFEST (`services replace`) and not `gcloud run deploy` flags: the container
command is
11     a /bin/bash wrapper carrying shell metacharacters (pipes, redirects, &&) ? on Windows
the gcloud
12     cmd-shim mangles those through `--command/--args`, while a YAML round-trips them
verbatim. Run
13     gcloud from PowerShell, not git-bash (MSYS rewrites `/tmp`-style paths).
14
15     The app is configured through a config.json baked INTO the manifest: the wrapper
base64-decodes
16     it to `/tmp/apphome/config.json` and exports `RALLY_APP_HOME=/tmp/apphome` before
exec'ing the
17     server, so one image serves every config shape (config.json wins over the cloud-serving
preset ?
18     backend/config/loader.py). `--edge-auth` is REQUIRED and explicit: `off` is only for the
19     identity-token smoke/e2e window; redeploy with `on` (the Cf-Access JWT verify, M9)
before the
20     service faces testers again.
21
22     Deliberate manifest choices (learned on the A1 deploy):
23     * `run.googleapis.com/cpu-throttling: "false"` + minScale 1 ? the in-process job
worker drains
24     on a background thread; without always-allocated CPU a 202'd compile stalls (README
§3a).
25     * maxScale DEFAULTS TO 1 ? the jobs/videos DB is per-instance ephemeral SQLite, so a
second
26     instance splits the queue and `/api/jobs/{id}` polls (issue #471, option 1). Raise
it only
27     once state is shared.
28     * `CLOUD_RUN_JOB` is NOT set ? it's a reserved runtime env name and the manifest is
```

```

rejected
29     with it; the code default ("rally-worker") is already correct.
30     * No secrets here: bucket/project/aud/team-domain are the same non-secret identifiers
the
31     committed runlogs carry; the Gemini key stays in Secret Manager (the alpha runs
32     `skip_ai_handoff=true` and never needs it).
33     ""
34
35     from __future__ import annotations
36
37     import argparse
38     import base64
39     import json
40     import sys
41
42     DEFAULT_PROJECT = "gen-lang-client-0306026826"
43     DEFAULT_REGION = "asia-southeast1"
44     DEFAULT_SERVICE = "rally-control-plane"
45     DEFAULT_SERVING_BUCKET = "gs://khelesutra-rally-serving"
46     DEFAULT_CF_TEAM_DOMAIN = "https://khelesutra.cloudflareaccess.com"
47     DEFAULT_CF_ACCESS_AUD =
"c980da67ed888fff935fcd913ae1eb4da63f9d65805d6e1c5ca76f92f517082b"
48
49
50     def build_config(args: argparse.Namespace) -> dict:
51         ""The app config the wrapper materializes at RALLY_APP_HOME/config.json.""
52         bucket = args.serving_bucket.rstrip("/")
53         return {
54             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
55             # WASB-only alpha: candidate windows ARE the rallies; no Gemini key in this
service.
56             "indexing": {"skip_ai_handoff": True},
57             "storage": {
58                 "backend": "gcs",
59                 "output_root": bucket,
60                 "input_backend": "gcs",
61                 "input_root": f"{bucket}/videos",
62             },
63             "security": {
64                 "edge_auth_enabled": args.edge_auth == "on",
65                 "cf_team_domain": args.cf_team_domain,
66                 "cf_access_aud": args.cf_access_aud,
67                 "allowed_video_root": "/tmp/apphome/output",
68             },
69         }
70
71
72     def build_yaml(args: argparse.Namespace) -> str:
73         ""Render the Knative service manifest for `gcloud run services replace`.""
74         cfg_b64 = base64.b64encode(
75             json.dumps(build_config(args), separators=(",", ":")).encode("utf-8")
76         ).decode("ascii")
77         # mkdir BEFORE the redirect (the redirect can't create the directory), then exec so
the
78         # server is PID 1 and Cloud Run's SIGTERM reaches it directly.
79         wrapper = (
80             "mkdir -p /tmp/apphome && "
81             f"printf %s '{cfg_b64}' | base64 -d > /tmp/apphome/config.json && "
82             "export RALLY_APP_HOME=/tmp/apphome && "
83             "exec python3 -m backend.main --server --host 0.0.0.0 --port 8080"
84         )
85         return f"""\
86 apiVersion: serving.knative.dev/v1
87 kind: Service
88 metadata:

```

```

89     name: {args.service}
90     labels:
91         cloud.googleapis.com/location: {args.region}
92 spec:
93     template:
94         metadata:
95             annotations:
96                 autoscaling.knative.dev/minScale: "{args.min_instances}"
97                 autoscaling.knative.dev/maxScale: "{args.max_instances}"
98                 run.googleapis.com/cpu-throttling: "false"
99     spec:
100         timeoutSeconds: {args.timeout}
101         containers:
102             - image: {args.image}
103               command: ["/bin/bash"]
104               args: ["-c", "{wrapper}"]
105               ports:
106                 - containerPort: 8080
107             resources:
108                 limits:
109                     cpu: "{args.cpu}"
110                     memory: {args.memory}
111             env:
112                 - name: DEPLOYMENT_PROFILE
113                   value: cloud-serving
114                 - name: CLOUD_RUN_REGION
115                   value: {args.region}
116                 - name: GOOGLE_CLOUD_PROJECT
117                   value: {args.project}
118                 - name: RALLY_ALLOWED_HOSTS
119                   value: "*"
120 ""
121
122
123 def main(argv: "list[str] | None" = None) -> int:
124     p = argparse.ArgumentParser(description=__doc__.splitlines()[0])
125     p.add_argument(
126         "--edge-auth",
127         choices=("on", "off"),
128         required=True,
129         help="REQUIRED and explicit: 'off' only for the identity-token smoke/e2e window;
130 "
131         "redeploy 'on' (M9 Cf-Access verify) before testers touch it again.",
132     )
133     p.add_argument("--project", default=DEFAULT_PROJECT)
134     p.add_argument("--region", default=DEFAULT_REGION)
135     p.add_argument("--service", default=DEFAULT_SERVICE)
136     p.add_argument(
137         "--image",
138         default=None,
139         help="Container image; default = the project's rally-worker:latest (post-#470 it
140 "
141         "carries cloud-run + auth, so the worker image IS the control-plane image).",
142     )
143     p.add_argument("--serving-bucket", default=DEFAULT_SERVING_BUCKET)
144     p.add_argument("--cf-team-domain", default=DEFAULT_CF_TEAM_DOMAIN)
145     p.add_argument("--cf-access-aud", default=DEFAULT_CF_ACCESS_AUD)
146     p.add_argument("--min-instances", type=int, default=1)
147     p.add_argument(
148         "--max-instances",
149         type=int,
150         default=1,
151         help="Default 1: per-instance SQLite state splits across instances (issue
152 #471).",
153     )

```

```

151     p.add_argument("--cpu", type=int, default=2)
152     p.add_argument("--memory", default="4Gi")
153     p.add_argument("--timeout", type=int, default=300)
154     p.add_argument("-o", "--output", default="-", help="Output path, or '-' for
stdout.")
155     args = p.parse_args(argv)
156     if args.image is None:
157         args.image = f"{args.region}-docker.pkg.dev/{args.project}/rally/rally-
worker:latest"
158
159     text = build_yaml(args)
160     if args.output == "-":
161         sys.stdout.write(text)
162     else:
163         with open(args.output, "w", encoding="utf-8", newline="\n") as f:
164             f.write(text)
165     return 0
166
167
168 if __name__ == "__main__":
169     raise SystemExit(main())
170

```

41. assistant (2026-07-05T06:56:50.073Z)

Now I have decisive evidence. The manifest:

- region `asia-southeast1`, memory `4Gi`, no HTTP/2 (`run.googleapis.com/use-http2` annotation absent ? default HTTP/1).
- The wrapper writes config to `/tmp/apphome/config.json`, `RALLY\_APP\_HOME=/tmp/apphome`. Uploads land under `/tmp/apphome/output/uploads` ? ****RAM-backed tmpfs on the 4Gi instance****.
- `allowed\_video\_root: /tmp/apphome/output`.

Let me read the README section that explicitly discusses tmpfs sizing and the 5.8 GB upload ? this is the authors' own statement of the memory-vs-upload constraint, and it clarifies whether the intended path is the free-space 507 or something else.

42. user (2026-07-05T06:56:51.502Z)

```

1      # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3      The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4      dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5      a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7      > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8      > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9      > cost first, **delete every resource after** ([gcp-vm-always-delete]). CI does
**not** build this image.
10
11     ## 0. Prerequisites (owner)
12     - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13     - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14     `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15     - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17     ## 1. Build + push the image (Artifact Registry)
18     ```bash
19     PROJECT=<gcp-project>; REPO=rally

```

```

20 # L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
21 REGION=asia-southeast1
22 gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
23>/dev/null || true
24 # `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
25 gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
26 ```
27 *(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per
D17.)*
28
29 ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
30 The proven shape (first real M7 deploy, 2026-07-03): weights staged in a **same-region
bucket**
31 mounted read-only via GCS FUSE (no weights in the image ? WASB-C3; no init-fetch step
needed).
32 ```bash
33 gcloud run jobs create rally-worker \
34 --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
35 --region $REGION --gpu 1 --gpu-type nvidia-l4 --no-gpu-zonal-redundancy \
36 --cpu 8 --memory 32Gi \
37 --max-retries 0 --task-timeout 3600 \
38 --set-env-vars WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar \
39 --add-volume name=serving,type=cloud-storage,bucket=$SERVING_BUCKET \
40 --add-volume-mount volume=serving,mount-path=/gcs
41 ```
42 - **`--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it
also halves
43 the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive
Y/n which,
44 in a non-interactive shell, **hangs forever with zero output and nothing created**. An
empty
45 `gcloud` that "did nothing" ? suspect a hidden prompt.
46 - 8 vCPU / 32 GiB: decode + x264 are CPU-side (NVDEC is a deferred D16 knob), so CPU
headroom is
47 what keeps the paid GPU busy. `/tmp` is **RAM-backed** ? the pulled input video lives
in memory,
48 so size `--memory` for the largest expected upload (a 5.8 GB GoPro file ? 6 GiB of
tmpfs).
49
50 ## 3. Wire the dispatcher (control plane)
51 The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
52 `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
53 ```bash
54 export DEPLOYMENT_PROFILE=cloud-serving
55 export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
56 GOOGLE_CLOUD_PROJECT=$PROJECT
57 # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
58 # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
59 python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
60 ```
61
62 **The AI handoff needs its key IN THE JOB env** (`GEMINI_API_KEY` ? mount a Secret
Manager secret
63 on the job), or run fully-local with `--set indexing.skip_ai_handoff=true` (candidate
windows
64 emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to
a silent
65 0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail
this at
66 boot (`AI_HANDOFF_KEY_MISSING`, rc 2) before the video is even pulled.
67

```

```

65     ### 3a. Control-plane SERVICE deploy ? required flags for reliable async jobs
66
67     The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-
PROCESS**: the
68     request returns `202` and a background thread drains the job (GPU detection dispatches
to the Job;
69     the reel stitch runs on the control plane itself). On Cloud Run that background thread
only makes
70     progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:
71
72     **The canonical deploy path is the committed manifest generator** (#473) ? it bakes the
73     cloud-serving config.json into a startup wrapper, sets every flag below, pins
`maxScale=1`
74     (per-instance SQLite state, issue #471), and deliberately does NOT set `CLOUD_RUN_JOB`
(a
75     reserved runtime env name that gets a manifest rejected; the code default is already
right):
76     ```bash
77     python deploy/cloudrun/gen_service_yaml.py --edge-auth on -o service.yaml
78     gcloud run services replace service.yaml --region $REGION # from PowerShell on Windows
79     ```
80     `--edge-auth off` is only for the pre-CF identity-token smoke/e2e window; redeploy with
`on`
81     before testers touch the service again. The flag-based equivalent (for reference ? a
manifest
82     round-trips the bash wrapper's metacharacters, flags don't on Windows):
83     ```bash
84     gcloud run deploy rally-control-plane --image <image> --region $REGION \
85     --no-cpu-throttling \           # CPU stays allocated between requests ? the drain thread
finishes
86     --min-instances 1 \           # one always-warm instance drains reliably (cheap CPU;
NOT the D7 GPU pool)
87     --cpu 2 --memory 4Gi --port 8080 --allow-unauthenticated \
88     --set-env-vars DEPLOYMENT_PROFILE=cloud-
serving,CLOUD_RUN_REGION=$REGION,GOOGLE_CLOUD_PROJECT=$PROJECT
89     ```
90     Without `--no-cpu-throttling`, a `/api/compile` returns 202 and then **stalls** ? the
reel never
91     finishes and startup crash-recovery would mark it terminal. `min-instances 1` does NOT
contradict
92     D7 (that was the ~$500/mo warm **GPU** pool); this is a ~1-vCPU CPU instance. Defense-
in-depth is in
93     the code: `Database.recover_stale_jobs()` **re-queues** a compile interrupted by a
restart (idempotent
94     stitch) rather than failing it, so even an unlucky scale-down mid-stitch self-heals on
the next drain.
95     Reserve `--allow-unauthenticated` for the pre-CF-auth smoke test only; lock ingress to
the CF proxy
96     (M9) before inviting testers. The image ships the `auth` extra (PyJWT[crypto]), so
flipping
97     `security.edge_auth_enabled` on needs no derived image ? the M9 Cf-Access JWT verify
works out of
98     the box.
99
100    ## 4. M7 acceptance (the owner runlog)
101    Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
102    `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
103    rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
104    exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
105
106    ## Gotchas
107    - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`

```



```

108      + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
109      - Completion is a bucket marker (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
110      ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
111      - A failed Job (validation rc 2 / segmenter rc 3) now writes a failure marker
carrying the
112      real non-zero rc, so the dispatcher's poll terminates FAST + accurately (no 30-min
hang on a bad
113      video). Only a hung/crashed container (no marker at all) makes the bounded poll
time out
114      (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean rc 4, and
the API path
115      surfaces it as a `cloud dispatch error` job failure (not a traceback).
116      - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
117      - GOOGLE_CLOUD_PROJECT is required for the real client ?
`GoogleCloudRunJobClient.run_job`
118      fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real
`job_path`;
119      an unset project would otherwise build a bogus `projects/None/...` name). Export it
(step 3) before
120      dispatching. The guard runs before the `google-cloud-run` import, so it is exercised
in CI even
121      though the SDK isn't a CI/test dependency.
122      - Hand-rolled `jobs execute --args` with `gs://` URIs: gcloud's default comma-split
plus its
123      alternate-delimiter syntax both bite ? repeated `--set` tokens trip the comma parser,
and a
124      `^:` delimiter splits the `:` inside `gs://`. Use a delimiter that appears nowhere in
the args,
125      e.g. `--args='^@@^-v@@infer@@--video-uri=gs://?'`. (The dispatcher path doesn't have
this
126      problem ? prefer it; this is for one-off diagnostics.) Container-arg overrides must
include the
127      dispatcher's two forced `--set`s: `deployment.compute_target=local_gpu`,
128      `indexing.detector_impl=native`.
129      - Git Bash (MSYS) on Windows mangles Unix-looking values before gcloud sees them:
130      `--add-volume-mount ?mount-path=gcs` became `C:/Program Files/Git/gcs`, and
131      `--command=/bin/bash` became `C:/Program Files/Git/usr/bin/bash` (the container then
exec-fails).
132      Run gcloud commands with such flags from PowerShell (`MSYS_NO_PATHCONV=1` breaks
gcloud's own
133      wrapper). PowerShell has its own trap: quote any flag value containing commas.
134      - First execution after an image push spends minutes in "Container image import in
progress"
135      (visible in `executions describe status.conditions`, NOT in logs) ? that's progress,
not a hang;
136      later executions reuse the imported image.
137

```

43. assistant (2026-07-05T06:57:05.048Z)

I now have all the evidence I need. Let me synthesize.

The control-plane **service** (4Gi RAM, the one the browser uploads to) writes the assembled file to `/tmp/apphome/output/uploads` ? RAM-backed tmpfs. The README's own words (§2, about the JOB but the same tmpfs mechanic applies): "`/tmp` is RAM-backed ? size `--memory` for the largest expected upload." The service is only 4Gi.

Now the exact 413 mechanism. Let me confirm the definitive fact about Cloud Run's request-size cap, which is the piece that makes a 95 MB part 413 before the app runs. Cloud Run's documented **request** limit: 32 MiB for HTTP/1 requests (no streaming). This manifest does not set `run.googleapis.com/use-http2`, so ingress is HTTP/1, and each 95 MB PUT part carries a fixed 95

MB Content-Length ? a buffered (non-streamed from the client's perspective) request that the Google Front End rejects with ****413**** before FastAPI's ``upload_part`` ever runs.

Let me do the final part-count arithmetic and double check the client sends fixed Content-Length parts (it does ? ``body: chunk`` is a Blob, fetch sets Content-Length).

2749 MB / 95 MB = 28.9 ? 29 parts, each ? 95 MB. Part 0 is 95 MB > 32 MiB ? ****413** at the Cloud Run ingress on the very first part**.

I have enough to answer with high confidence. Let me record the precise evidence lines and produce the structured output.

44. user (2026-07-05T06:57:06.517Z)

```
95     segmenters?: string[];
96     default_segmenter?: string;
97     ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?:
string };
98     gpu?: { nvidia_smi_present?: boolean };
99     /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step.
*/
100    gemini_key_present?: boolean;
101    wasb_weights_present?: boolean;
102    /** The largest upload THIS box accepts, in MB (engine `security.max_upload_mb`,
default 4096).
103    * /upload/init already 413s an oversized file BEFORE any part transfers; the SPA
consumes this to
104    * reject it INSTANTLY client-side (no wasted round-trip). Absent on older engines ?
the guard is
105    * skipped and the engine's server-side fail-fast still applies. */
106    max_upload_mb?: number;
107    /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch
(a Cloud Run
108    * GPU Job)? The ingest compute-picker shows the "in the cloud" option only when
true. */
109    deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
110    /** Which storage media THIS box can store INTO (P3). The medium picker renders only
`usable`
111    * backends; `free_gb` is `null` for a cloud bucket (no readable capacity). Absent on
older engines
112    * ? the SPA degrades to local-only (no picker). */
113    storage?: StorageCapabilities;
114    [key: string]: unknown;
```

45. assistant (2026-07-05T06:57:11.098Z)

Confirmed. The SPA's ``types.ts`` declares ``max_upload_mb?`` as an OPTIONAL top-level field on ``EngineCapabilities``, but the current-master engine ``/api/capabilities`` (backend/main.py:379-405) does NOT emit it. So even on current master the client-side pre-flight guard is inert (``maxUploadBytes`` stays ``undefined``), and on the deployed old build it's likewise absent. The browser always proceeds to chunk-and-upload.

I have a complete, evidence-backed trace. Producing the structured output.

46. user (2026-07-05T06:58:05.847Z)

Structured output provided successfully